ZK 专家养成课程 · 提高篇与论文精读（S07–S11）

> 本卷由 5 讲合并装订而成（内容与单文件版逐字一致）。篇内各讲以 `# S/L/M xx` 标题分界，总纲见 `README_课程总纲.md`。

本卷目录

- **S07** · S07 · 承诺：给整张表发一枚指纹
- **S08** · S08 · 收口：从几十条主张到一次打开
- **S09** · S09 · 查表：非线性函数的证明
- **S10** · S10 · 组装：一台完整的机器（GKR 范式与 DeepProve 全景）
- **S11** · S11 · 论文精读：DeepProve（逐节伴读）

S07 · 承诺：给整张表发一枚指纹

> S06 结束时有一条悬在空中的主张："f(7,4) = 11"。验证方手里没有表，凭什么信？这一讲造出那枚"指纹"——你先看到两种失败的设计（包括亲手破解其中一种），看到修复它需要怎样一个"魔法世界"，然后**用纯多项式算术**亲手完成一次"打开"。 前置：S03（MLE）、S04（eq）、S06（sumcheck 全程）。延续 mod 13 与表 [3,1,2,0]。

---

1. 先明确要什么

协议开始前，证明方把表交给"指纹机器"，得到一个**短**的数 C；此后任何"表在点 r 取值为 y"的主张，都用 C 核销。三个要求照旧（S01）：C 短；诚实者必过；**改了表，指纹必然对不上**——最后这条是新的，叫**绑定性**。

2. 第一次尝试：仪式与"力量表"

直接哈希整张表？验证方核对指纹就得读全表——违背"短"。换个思路：

> 找一位**仪式主持人**，他心里藏一个秘密数 τ（仪式结束后销毁，谁也不许知道），然后公开一张"力量表"：τ 的各次幂。

在我们的 13 格钟里，设 τ = 6，力量表前 4 项（4 = 表长）：

```
w = [τ⁰, τ¹, τ², τ³] = [1, 6, 36≡10, 6×10≡8]
```

证明方承诺：**C = 表的每个数 × 对应力量，再全加**：

```
C = 3×1 + 1×6 + 2×10 + 0×8 = 3 + 6 + 20 = 29 ≡ 3 (mod 13)
```

关键性质：**证明方只需要力量表，不需要 τ**——他做的只是加权求和。（4 项的表体会不到，2^{20} 项时这就是一次"百万项加权求和"——S08 会告诉你硬件怎么跑。）

3. 亲手破解这个玩具

"改表不改指纹"的组合存在吗？试一对系数 δ = (6, −1, 0, 0)（即 δ ≡ (6, 12, 0, 0)）：

```
Σ δᵢ·wᵢ = 6×1 + (−1)×6 = 6 − 6 = 0 ✓
```

于是假表 f' = f + δ = [9, 0, 2, 0] 的指纹：

```
C' = 9×1 + 0×6 + 2×10 + 0×8 = 29 ≡ 3 = C
```

不同的表、同一枚指纹——玩具被破解。 看清 δ 的真面目：δ 对应多项式 δ(X) = 6 − X = −(X − 6) = **−(X − τ)**——它在 τ 处恰好为零，所以它乘任何数加进表都改不了 C。**知道 τ 的人，造这种多项式只要一秒**。

那不知道 τ 的攻击者呢？他要从公开的 w = [1, 6, 10, 8] 里找"加权为零的非零组合"。可惜在纯数字世界里这太容易了：**任何**两个位置都行——δ = (w_1, $-w_0$) 永远满足 $w_1 \cdot w_0 - w_0 \cdot w_1 = 0$。纯数字世界满地都是暗门。

4. 修复：搬进"能加不能开锁"的世界

修复思路：把力量表从"裸数字"换成**一种只能做加法组合、却无法反推其构造规律的量**。真实系统用椭圆曲线上的点：仪式主持人公开的不是 τ^i 这个数，而是**点** $W_i = \tau^i \cdot G$（G 是一条公开的"生成点"）。点可以相加、可以乘系数（这正是证明方算 $C = \Sigma f_i \cdot W_i$ 需要的全部能力），但从 W_1 和 W_0 反推 τ，等价于离散对数问题——几十年密码学攻不动的硬骨头。第 3 节的攻击在点世界里的版本是"找 $\delta_0 \cdot W_0 + \delta_1 \cdot W_1 = 0$(点)"，展开就是 $\delta_0 + \delta_1\tau \equiv 0$（在指数里）——**必须知道 τ**。暗门焊死了。

这套构造的名字：**KZG 多项式承诺**（2010 年提出；"对一张表的加密担保"）。承诺 $C = \Sigma f_i \cdot W_i$ 这次加权求和，在工程里就是一次 **MSM**——你已经在 FPGA 文档里见过它的算子规格（OP-10）。

边界注记（绑定 ≠ 隐藏）：本章的承诺只做到了**绑定**（表改不了，改了必被抓）；它并不**隐藏**表的内容——后续协议里披露的打开值、以及某些场景下的承诺本身都会泄漏关于表的信息。DeepProve 证明的是**完整性**（结果没被篡改），所以不需要隐藏；但若负载是隐私交易（M08 T-C），必须给承诺加盲化因子并重新审视零知识性——这是选型时最容易混为一谈的两个性质。

5. 打开：用"商多项式"证明一个点的取值

现在回答 S06 的悬案：V 持有 C，P 声称"f 在点 7 取值为 4"。怎么核?

先把表换成"承诺世界的身份"。**同一张表有两种读法**（这个切换是 HyperKZG 的核心，先记住用法）：S03 里表是"位 = 坐标"的 MLE；承诺世界里，表被读作**一元多项式的系数**:

```
u(X) = 3 + 1·X + 2·X² + 0·X³        （系数 = 表 [3,1,2,0]）
```

若 P 声称 u(7) = y，数学事实是:

```
u(X) − y  能被 (X − 7) 整除   ⟺   u(7) = y
```

（多项式在一点为零 ⟺ 含一次因子 (X−点)。）所以诚实的 P 可以拿出**商多项式** $q(X) = (u(X) - y)/(X - 7)$——一个次数低一档的多项式。P 对 u 的承诺早已存在（C）；他只需**再对 q 承诺一次**发给 V。V 不用知道 u 和 q 本体，就能核验"u − y 被 (X−7) 整除"这条恒等式在承诺世界成立——靠的是一种叫**配对**的运算（它让"点世界的乘法关系"可以被一次性检验；细节在阅读清单，本课要求你记住它保证的这一件事）。**只要 y 是假的，**

(u−y) 在 7 处不为零，整除不成立，任何 q 都凑不出这条恒等式——这就是绑定性兑现的地方。

这一整段你都可以亲手算（多项式代数不需要魔法世界）。先算 u(7)：

```
u(7) = 3 + 7 + 2×49 = 108 ≡ 4 (mod 13)
```

再用**综合除法**（系数 [2, 1, 3−4≡12]，除以 X−7，即逐位乘 7 再加）：

```
2      →  1+2×7=15≡2   →  12+2×7=26≡0  ✓ 余数 0（u(7)=4 确认）
商 q(X) = 2X + 2
```

验算（乘回去）：(2X+2)(X−7) = 2X² − 12X − 14 ≡ 2X² + X + 12 = u(X) − 4 √。

负例（亲手做一遍才懂"绑定性"卡在哪）：若 P 谎称 u(7) = 5，则 u−5 = 2X²+X+11，综合除法：2 → 1+14≡2 → 11+14≡25≡**12 ≠ 0**——余数非零，**不存在**能通过核验的 q。谎报一个点值 = 无商可除 = 必败。

6. 代码对应（简短）

- 力量表 = SRS 的 `g1_powers`；承诺 = `HyperKZG::commit`（dp-crypto `arkyper/mod.rs:457`，MSM）；
- 商多项式的综合除法 = `compute_witness_polynomial`（`arkyper/mod.rs:157-183`，递推 h[i−1] = b[i] + h[i]·u）；
- 配对核验 = `kzg_verify_batch`（:237-320）；
- "两种读法"在代码里的影子：`eval_as_univariate`（`dense.rs:765`，把 MLE 表当一元多项式在标量点求值）。

7. 动手作业

1. 仪式换 τ = 5：写力量表前 4 项（mod 13），算表 [3,1,2,0] 的承诺 C，并用 δ = (w_1, $-w_0$) 造一张同指纹的假表，验证 C' = C。（答案：w = [1, 5, 25≡12, 5·12≡8]；C = 3×1+1×5+2×12 = 3+5+24 = 32 ≡ 6；δ = (5, −1≡12, 0, 0)，f' = [8, 0, 2, 0]，C' = 8 + 2×12 = 32 ≡ 6 √。）
2. 对 u(X) = 3+X+2X² 在点 9 做打开：算 u(9)、综合除法、验算乘回。（u(9) = 3+9+162 = 174 ≡ 174−169 = 5；商 2X+3：余数 12+3·... 自己跑完。）

3. 思考：第 3 节的破解里，δ = (w_1, $-w_0$) 对应的多项式是什么？它为什么在 τ 处为零？（$w_1 - w_0X = \tau - \tau X = -\tau(X-1)$？——不对，代入 X=τ：τ − τ·τ = τ(1−τ)，一般非零——**做出来你会发现它不为零**，真正的组合是第 3 节那对 (6, −1) = −(X−τ)，它用到了 τ 本身。请解释：攻击者不知道 τ 时，为什么"随便挑两个位置"不再必然成功？）
4. Rust：用 `ark_bn254::{Fr, G1}` 实现玩具仪式（τ 随机、公开点、commit、打开的商多项式部分），与手算对拍。

做完放入 `homework/s07/`。

S08 · 收口：从几十条主张到一次打开

> 真实证明的结尾不是一条主张，是**几十条**：线性层吐出"权重在某点等于某值"，查表层吐出"witness 切片在某点等于某值"，边界表、偏置……每条都指向**不同的表、不同的点**。逐条打开承诺？打开一次的代价是配对级的——几十次就把验证方拖垮。这一讲你亲手把一整把主张**收口**成一次打开：三个步骤，每步都有完整的 mod-13 手算。 前置：S04（eq）、S06（sumcheck，含折叠）、S07（承诺、商多项式、两种读法）。

---

1. 问题：两把主张，两个点

延续我们的演员表：

- 表 f = [3,1,2,0]（已承诺），主张 ①：f(7,4) = 11（S06 的终点，点 (7,4)）；
- 表 g = [3,0,0,0]（已承诺，即"只有左上角是 3"），主张 ②：g(5,4) = 10（验算：g = 3(1−x)(1−y)，代 (5,4)：3(−4)(−3) = 36 ≡ 10 √）。

两把主张、**两张表、两个不同的点**。目标：只用**一次**承诺打开核销两把。

2. 第一步：把不同点的主张搬到同一个点（batched sumcheck）

思路：抽两个随机系数 γ_1、γ_2（S05 的老朋友——随机线性组合），构造一个**新的求和主张**：

```
Σ_{x∈{0,1}²} [ γ₁·eq((7,4), x)·f(x) + γ₂·eq((5,4), x)·g(x) ]  =  γ₁·11 + γ₂·10
```

右边 = V 自己能算的数（主张值是公开的）。左边为什么等于它？**逐项用 S04 的恒等式**：Σ_x eq((7,4),x)·f(x) = f(7,4) = 11 √，g 项同理。两张表的"探针恒等式"被 γ 加权捆成一个 sumcheck——**而 sumcheck 你在 S06 已经会跑了**。

取 $\gamma_1 = 3$, $\gamma_2 = 5$。右边（claimed sum）= 33 + 50 = 83 ≡ 5 (mod 13)。

跑 sumcheck。新东西只有一个：表达式里现在有**两张已知来源的表**（eq 表——V 自己会造，S04 教过倍增法）——所以轮函数是"两个一次式相乘再加"，是 **X 的二次多项式**，每轮要发 **3 个点**（二次式由 3 点确定；S06 §6 的"d+1 个点"在这里兑现，d=2）。

需要的四张表（S04 作业已备好）：

```
eq((7,4),·) = [5, 5, 2, 2]      eq((5,4),·) = [12, 11, 10, 7]
f = [3, 1, 2, 0]                g = [3, 0, 0, 0]
```

第 1 轮（削变量 x）。轮函数 $R(X) = \Sigma_y [\gamma_1 \cdot eq_1(X,y)f(X,y) + \gamma_2 \cdot eq_2(X,y)g(X,y)]$，逐点算 X=0,1，再推 X=2：

```
X=0: γ₁(5×3 + 2×2) + γ₂(12×3 + 10×0) = 3×19 + 5×36 = 57+180 ≡ 5+11 = 16 ≡ 3
X=1: γ₁(5×1 + 2×0) + γ₂(11×0 + 7×0) = 15 ≡ 2
X²系数 = γ₂·Σ_y Δeq₂·Δg = 5×[(11−12)(0−3) + (7−10)(0−0)] = 5×3 = 15 ≡ 2
（γ₁ 的 Δeq₁≡0 → 无二次项）
     （乘积的 X² 系数 = ΔA·ΔB，X 系数 = A₀ΔB + ΔA·B₀——两次式相乘的展开，S06 §6 的"度 d"来源）
R(X) = 3 + 10X + 2X²
```

消息 [R(0), R(1), R(2)] = [3, 2, 5]。V 查：R(0)+R(1) = 3+2 = 5 = claimed sum √。掷 **$r_1 = 6$**：

```
R(6) = 3 + 60 + 2×36 = 135 ≡ 5          （V 用三点插值核对：
3×L₀(6)+2×L₁(6)+5×L₂(6) = 30+4+10 = 44 ≡ 5 ✓）
```

第 2 轮（削变量 y）。四张表各自折叠（S06 的折叠，权重 (1−6, 6) ≡ (8, 6)）：

```
f' = [8×3+6×1, 8×2+6×0] = [30,16] ≡ [4, 3]      eq₁' = [8×5+6×5, 8×2+6×2] = [70,28] ≡ [5, 2]
```

```
g' = [8×3+6×0, 0]        ≡ [11, 0]              eq₂' = [8×12+6×11,
8×10+6×7] = [162,122] ≡ [6, 5]
```

轮函数 $R_2(Y)$（同样方法，Y=0: 3×5×4+5×6×11 = 60+330 = 390 ≡ **0**；Y=1: 3×2×3+0 = 18 ≡ **5**；Y^2系数 = 3×(2−5)(3−4) + 5×(5−6)(0−11) = 9+3 = 12）：

```
R₂(Y) = 6Y + 12Y²       （由 R₂(0)=0、R₂(1)=5、二次系数 12 联立）
消息 [R₂(0), R₂(1), R₂(2)] = [0, 5, 8]
```

V 查：0 + 5 = 5 = R(6) √。掷 **r_2 = 2**。

收口点到了。V 现在检查（**eq 的值 V 自己算**——他知道两个主张点，eq 是他的多项式，无需承诺）：

```
最终恒等式：γ₁·eq((7,4),(6,2))·f(6,2) + γ₂·eq((5,4),(6,2))·g(6,2) = 8
  eq((7,4),(6,2)) = eq(7,6)×eq(4,2) = [7×6+(−6)(−5)]×[4×2+(−3)(−1)] = 72×11
≡ 7×11 = 77 ≡ 12
  eq((5,4),(6,2)) = eq(5,6)×eq(4,2) = [50≡11]×11 = 121 ≡ 4
→ 3×12×f(6,2) + 5×4×g(6,2) = 8
```

两张表在同一个新点 (6,2) 上的取值，成了两条新主张：f(6,2) = 2、g(6,2) = 2（验算：f(6,2) = 3−12−2 = −11 ≡ 2 √；g(6,2) = 3(−5)(−1) = 15 ≡ 2 √）。**代回去：3×12×2 + 5×4×2 = 72+40 = 112 ≡ 8 √ 与 sumcheck 终值一致**。

一个必须写对的细节：sumcheck 过程中**不要**把 eq⊙f 预先合成一张表再去折叠——合成表折叠得到的是"乘积表的 MLE 在终点的值"，与"多项式乘积在终点的值"在格子之外**不相等**（本项目 early 审计时亲手算错过这一步：合成表折叠给 5，正确协议给 8）。正确做法：**各表独立折叠，轮函数每轮由各表现算合成**——这也是代码里"虚拟多项式"把表分开登记的原因。

3. 第二步：两张表塌成一张（RLC）

现在两条主张**同点不同表**。再抽一对系数（比如就用 3 和 5），构造随机线性组合：

```
h = 3×f + 5×g = [9+15, 3, 6, 0] = [24, 3, 6, 0] ≡ [11, 3, 6, 0]
```

于是"两把主张"变成"一把"：**h(6,2) = 3×2 + 5×2 = 16 ≡ 3**。验算（S03 的角展开）：h(6,2) = 11×96 + 3×72 + 6×16 = 1368 ≡ 3 √（1368 = 13×105 + 3）。只要 γ 是 V 掷的，

伪造一张 h' 同时满足"h'(6,2) = 3 且 h' 与真表只在无关紧要处不同"的概率是 2^{-127} 级——两张表的命运被随机捆死在一条多项式上。

4. 第三步：一次打开（折叠 + 三点）

最后用 S07 的承诺核销"h 在 (6,2) = 3"。这是**多维点**，而 KZG 的商多项式只会开**一元点**——桥就是**折叠**：

同一张表的两种身份在此合流。把表 [3,1,2,0] 读作一元系数 u(X) = 3+X+2X²（S07），逐变量折叠系数表：

```
折 x=7：p₁[j] = 7×(p[2j+1]−p[2j]) + p[2j] → p₁ = [7(1−3)+3, 7(0−2)+2] =
[−11,−12] ≡ [2, 1]
折 y=4：p₂ = 4×(p₁[1]−p₁[0]) + p₁[0] = 4×(1−2)+2 = −2 ≡ 11
```

两个惊喜（都是必然，不是巧合）：

1. p_1 = [2,1] 正是 **S06 第 3 节折叠出的表** f(7,·)——承诺世界的折叠与 sumcheck 世界的"固定一个变量"是**同一个操作**，这就是"两种身份"能共存的原因；
2. p_2 = 11 **正好是 S06 的最终主张 f(7,4) = 11**。整门课到这里闭环：sumcheck 的终点、折叠链的终点、承诺打开的点，是同一个点、同一个数。

三点技巧（省掉逐个打开）。逐个打开 p_1、p_2 要两次配对级核验；HyperKZG 的办法：V 抽随机 r，只要求 P 提供每张折叠表在 **r、−r、r²** 三个点的取值，然后用一条恒等式把它们串起来：

```
2r·p₁(r²)  =  r·(1−x)·(p₀(r)+p₀(−r))  +  x·(p₀(r)−p₀(−r))
```

亲手验证（r = 5，x = 7）：u(5) = 3+5+50 = 58 ≡ 6；u(−5) = u(8) = 3+8+128 = 139 ≡ 9；$p_1(X)$ = 2+X → $p_1(25\equiv12)$ = 14 ≡ 1。

```
左边 = 2×5×1 = 10
右边 = 5×(1−7)×(6+9) + 7×(6−9) = 5×(−6)×15 + 7×(−3) = −450−21 = −471 ≡
−471+13×37 = 10 ✓
```

为什么这条恒等式成立（两行推导，自己补全）：把 u 拆成偶/奇两组系数 u(X) = A(X²) + X·B(X²)，则 u(±r) 给出 A(r²) 与 B(r²)，而 $p_1(X)$ = (1−x)A(X) + x·B(X)——代进去整理即得。它的威力：**V 从"已承诺的 u 在 ±r 的取值"外推出"未单独承诺的 p_1 在 r² 的取值"**，

于是所有折叠表只需要一次批量打开（3 个点、1 次 MSM + 1 次配对核验）——20 个变量的表：19 张折叠表逐个打开要 19 次核验，三点法只要 1 次。

5. 收口全景（把 S01 的游戏正式结案）

```
几十条主张（不同表、不同点）
  →① batched sumcheck：γ 加权 + 各自 eq 表进同一个 sumcheck
      → 所有表在同一个新点 r* 的取值（V 自己补 eq 部分）          【本讲 §2】
  →② RLC：γ' 加权 → 一条多项式 h                                  【本讲 §3】
  →③ HyperKZG：折叠链 + 三点恒等式 + 一次批量打开                  【本讲 §4】
几十次核验  →  1 次
```

代码对应：① = `claims_batching_sumcheck`（`poly_commit/prover.rs:140`，eq 表与表达式构造 `poly_commit/mod.rs:83`；验证侧闭合等式在 `poly_commit/verifier.rs`，精读时自查）；② = `linear_combination`（prover.rs:115-131）；③ = `HyperKZG::open`（`arkyper/mod.rs:329`：折叠：343-359、三点 :373、批量打开 :186-234）。本讲的 §2 例子就是那个真实流程在 mod 13 的完整缩影——真实系统只是把 2 条主张换成上百条、把 13 换成 2^{254}、把 2 个变量换成 20 个。

6. 动手作业

1. 把 §2 的 batched sumcheck 换挑战重跑：$r_1 = 3$、$r_2 = 9$（γ 不变）。要求列出两轮消息与终值，并用"V 插值"复核 R(3)。（自检锚点：第 1 轮消息不变 [3,2,5]；R(3) = 3+30+18 = 51 ≡ 12；折叠后 eq_1' = [8×5+6×5? ——注意权重是 (1−3, 3) = (−2, 3) ≡ (11, 3)：eq_1' = [11×5+3×5, 11×2+3×2] = [70, 28] ≡ [5,2]…… 请自己算完，终值应满足 $3\times eq_1((7,4),(3,9))\times f(3,9) + 5\times eq_2((5,4),(3,9))\times g(3,9)$ = 终值。）
2. §4 的三点恒等式换 r = 3 再验一次（x = 7，p_0 = [3,1,2,0] 读作 u）。（锚点：u(3) = 3+3+18 = 24 ≡ 11；u(−3) = u(10) = 3+10+200 = 213 ≡ 213−208 = 5；$p_1(9)$ = 2+9 = 11；左边 = 2×3×11 = 66 ≡ 1；右边 = 3×(−6)×16 + 7×6 = −288+42 = −246 ≡ −246+13×19 = 1 √。）
3. 证明"合成表折叠 ≠ 多项式乘积"：对 §2 的 $eq_1 \odot f$（γ_1=3）合成表 [6,2,12,0]，折叠 x=6 得 5；而多项式乘积 $\gamma_1 \cdot eq_1(6,y)\cdot f(6,y)$ 在 y=0,1 的和是 8。写出两者为何不同（提示：MLE($eq_1 \odot f$) 与 $eq_1 \cdot f$ 两个多项式只在布尔格上相等），并由此解释为什么实现必须"分表折叠、轮内合成"。
4. Rust：实现本讲的收口三步（toy 承诺可用 S07 作业的纯数字版），对拍全部中间数。

做完放入 `homework/s08/`。

S09 · 查表：非线性函数的证明

> 到目前为止你能证明"线性代数"：矩阵乘（sumcheck 的天职）、加法、加权。但神经网络的灵魂是非线性的——GELU、softmax、归一化、量化右移。它们**没有**低次多项式表达式，sumcheck 无从下口。这一讲造出处理它们的机器：**查表论证**。你将全程手算一个完整例子：一张 4 格的表、3 次查询、两侧分数折叠、最终交叉核对——最后才听到 LogUp 这个名字。 前置：S02（逆元）、S04（恒等式的用法）、S06（sumcheck 机制——本讲只借用"逐层折叠"的形状，不重跑）。

---

1. 非线性为什么难，以及"表"为什么是出路

GELU 之类的函数画出来是一条曲线。S03 告诉我们：一个多项式若在"足够多点"上与曲线重合，它就必须是曲线本身——而曲线不是多项式。**所以不存在能表达 GELU 的低次多项式**，S03–S08 的机器（全部以多项式为燃料）直接熄火。

出路：**离线把函数值打成一张表**。以量化世界的 GELU 为例：输入是整数格点，把每个可能的输入对应的输出预先算好存成表 T。运行时"计算 GELU"="查表"。证明任务随之变成：

> **证明：每次查询的值 q_i 确实在表 T 里，且报告的输出 o_i 确实是 T 给的那个值。**

表的规模只由**输入位宽**决定（16 位输入 → 约 6.5 万格），与数据量无关——这就是 zkML 系统能负担的原因。

2. 笨办法与它的致命伤

让 P 直接把所有 q_i 和 o_i 发给 V，V 逐个查表核对。正确，但通信量 = 查询条数 × 2——**违背"短"**（S01）。需要一个把"百万次查询"压缩成"一条短主张"的办法。

3. 关键构造：把每次查询变成一个分数

选一个随机数 β（作用马上讲），把**每一次查询**表示成分数：

```
查询 qᵢ  →  分数 1 / (β + qᵢ)
```

把**表的每个位置** j 也表示成分数：

```
表位置 j（值 T[j]）  →  分数 m_j / (β + T[j])        m_j 叫"多重度"
```

恒等式（本讲的心脏）：

```
查询侧总和 = 表侧总和
Σᵢ 1/(β+qᵢ)  =  Σⱼ mⱼ/(β+T[j])
```

为什么它等价于"查询全部合法"：把两侧都按"值"归组。查询侧 = Σ_值v c_v/(β+v)（c_v = 值 v 被查的次数）；表侧 = Σ_值v $t_v \cdot m_v'$/(β+v)（t_v = v 在表中占的位置数，m_v' = 每个位置的多重度）。两侧相等 ⟺ 对每个值 v：**$t_v \cdot m_v' = c_v$**——即"多重度恰好记录了每个值被查的次数"。若有一次查询查了表外的值（或次数被篡改），某个值的 c_v 对不上，两侧总和必然不同（除非 β 恰好撞进一个坏集合——β 是 V 掷的 128-bit 随机数，撞上 = 概率可忽略；这就是 β 的作用：把"值不同"放大成"分数和不同"）。

多重度怎么算：m_j = c_{T[j]} ÷ t_{T[j]}（被查次数 ÷ 表内出现次数——除法在钟表世界里是乘逆元，S02 的老手艺）。

4. 亲手跑一遍（mod 13，全程数字）

设定：表 T = [1, 2, 3, 4]（4 个位置，每个值出现一次 → t_v = 1）；查询序列 q = [2, 1, 1]（值 2 查一次、值 1 查两次）。β = 4（检查 β+v = 5,6,7,8 都不为 0 √）。

多重度：c(1) = 2, c(2) = 1, c(3) = 0, c(4) = 0 → m = [2, 1, 0, 0]。自检：Σm = 3 = 查询条数 √。

两侧直接对账（用 S02 的逆元：1/5 = 8，1/6 = 11，1/7 = 2，1/8 = 5）：

```
查询侧 = 1/(4+2) + 1/(4+1) + 1/(4+1) = 11 + 8 + 8 = 27 ≡ 1 (mod 13)
表侧   = 2/(4+1) + 1/(4+2) + 0 + 0   = 2×8 + 11     = 27 ≡ 1 ✓
```

两侧相等，查询合法 √。（作弊测试：若 P 谎称查询了值 9——表里没有——查询侧多出 1/(4+9) = 1/13？β+9 = 13 ≡ **0**，分母为零——这正是 β 要随机的另一个原因：它让"表外

的值"极大概率对应一个合法但**对不上账**的分数，而不是直接溢出。换成 β=4、谎查值 5：查询侧多 1/9 = 3（9×3 = 27 ≡ 1），总和变 4 ≠ 1 → 抓住 √。）

5. 不做除法的折叠：分数的 GKR 电路

第 4 节的对账用了**逐个求逆元**——6.5 万格的表就要 6.5 万次求逆，太贵。LogUp 的工程妙笔：**全程不做除法**。每个分数只记 (分子, 分母) 一对，分数相加用交叉相乘：

```
n₁/d₁ + n₂/d₂  =  (n₁d₂ + n₂d₁) / (d₁d₂)          （3 乘 1 加，无逆元）
```

把所有分数**两两折叠**成一棵树（形状与 S06 的表折叠一模一样），最后每侧剩一个 (分子, 分母)。V 核对两侧的方式也交叉相乘：

```
查询侧.分子 × 表侧.分母  =  表侧.分子 × 查询侧.分母
```

亲手跑（查询侧 3 个分数：(1,6), (1,5), (1,5)）：

```
折 (1,6)+(1,5) = (1×5+1×6, 6×5) = (11, 30≡4)
再折 (1,5)：    (11×5 + 1×4, 4×5) = (59≡7, 20≡7)
查询侧终值分数 = (7, 7)          → 值 = 7/7 = 1 ✓（与第 4 节一致）
```

表侧（分数：(2,5), (1,6), (0,7), (0,8)）：

```
(2,5)+(1,6) = (2×6+1×5, 30) = (17, 4)
(17,4)+(0,7) = (17×7, 28) = (119≡2, 28≡2)
(2,2)+(0,8) = (16, 16)
表侧终值分数 = (16≡3, 16≡3)      → 值 = 1 ✓
```

交叉核对（V 的全部工作）：

```
查询侧.分子 × 表侧.分母 = 7 × 3 = 21 ≡ 8
表侧.分子 × 查询侧.分母 = 3 × 7 = 21 ≡ 8    ✓ 相等 → 查表合法
```

无一次除法、无一次求逆。这棵"分数折叠树"逐层是一次数值计算——**又是 GKR 的分层形状**：每层一次 sumcheck（证明"下一层 = 本层折叠"），挑战点逐层传递，层数 = $\log_2$(表长)（4 格的表 2 层；6.5 万格 16 层——只随位宽涨，不随数据量涨）。

6. 还缺一块：把"表世界"焊回"张量世界"

第 3–5 节证明了"witness 里记录的查询/输出对与表一致"。但真正的目标是**节点的输出张量**。最后一块拼图叫 **linking sumcheck**：再用一次 sumcheck 证明"输出张量的 MLE = 查表输出按算子语义的组合"。各算子的组合方式：

算子	linking 的内容
GELU / requant	输出 = 主表查出的值（requant 的乘子与右移已折进表里）
GLU	输出 = 查表半 × 直通半（两半拼接）
softmax	输出 = 查值 − shift（shift 也是推理期记录的 witness）
RMSNorm / LN	输出 = 输入 ⊙ 归一化因子（因子本身是 witness，逐元素乘进 sumcheck）

至此查表层的完整证明链闭环：**witness 切片 + 多重度 → 承诺（S07）→ LogUp 双侧折叠（本讲 §5）→ linking（本讲 §6）→ 输入侧新主张回传上游（S06 的 claim 链）→ 收口（S08）**。

名字落点：这套"分数 + 折叠"的构造叫 **LogUp**；配 GKR 分层电路的实现叫 **LogUp-GKR**。DeepProve 论文 §4 的原话：*"lookup arguments based on LogUp for proving non-linear layers"*。

7. 代码对应

- witness 切片 + 多重度：`generate_lookup_witness_for_chunk`（`lookup/context.rs:233`，多重度公式在 :317 起——本讲 §3 的 m_j 公式）；
- 双侧批量折叠：`new_batch_multiple_sizes_prove`（`lookup/logup_gkr/prover.rs:91`）与层表达式 `circuit.rs`（三种层的 sumcheck 表达式——FPGA 文档 OP-13 §13.1-13.2 有完整推导与符号对照）；
- linking sumcheck：`prove_linking_sumcheck`（`lookup/operation/inputs/proving.rs:26`）；
- 验证方两侧交叉核对：`verifier.rs:587`（本讲 §5 的"分子×分母"检查）。

8. 动手作业

1. 表 T = [2, 1, 2, 3]（值 2 占两格！），查询 q = [1, 2, 2]。取 β = 5：算多重度 m、两侧分数和（直接法，含逆元）、两侧折叠（交叉相乘法）、交叉核对。自检：m = [0, 1, 2, 0]（c(1)=1, c(2)=2, t(2)=2 → 每格 1）；查询侧 = 1/6 + 2/7 = 11 + 2×2 = 15 ≡ 2；表侧 = 1/6 + 2×(1/7) + 1/8？——**注意第三格值是 2 不是 3**：m = [0,1,2,0] 对应位置值 [2,1,2,3]，表侧 = 0/(β+2) + 1/(β+1) + 2/(β+2) + 0/(β+3) = 1/6 + 2/7 = 11+4 = 15 ≡ 2 √。
2. 作弊实验：查询里混入一次"查表外值 5"（q = [2,1,5]，β = 4）。分别用直接法与折叠法指出哪里对不上。（锚点：查询侧多出 1/(4+5) = 1/9 = 3；两侧不再相等；折叠后交叉乘积不等。）
3. 把第 5 节的查询侧折叠画成树（3 个叶子 2 层），标出每层的 (分子, 分母)；说明层数 = $\lceil \log_2 \text{叶子数} \rceil$。
4. 思考：为什么表长只由位宽决定、不随推理数据量涨？什么量随数据量涨？（提示：查询条数 → 查询侧电路的叶子数与 witness 多重度之和。）
5. Rust：实现第 5 节的分数折叠核（(n,d) 对、无除法），对表 [1,2,3,4] 与任意查询序列输出两侧终值分数并交叉核对；再做负例。

做完放入 `homework/s09/`。

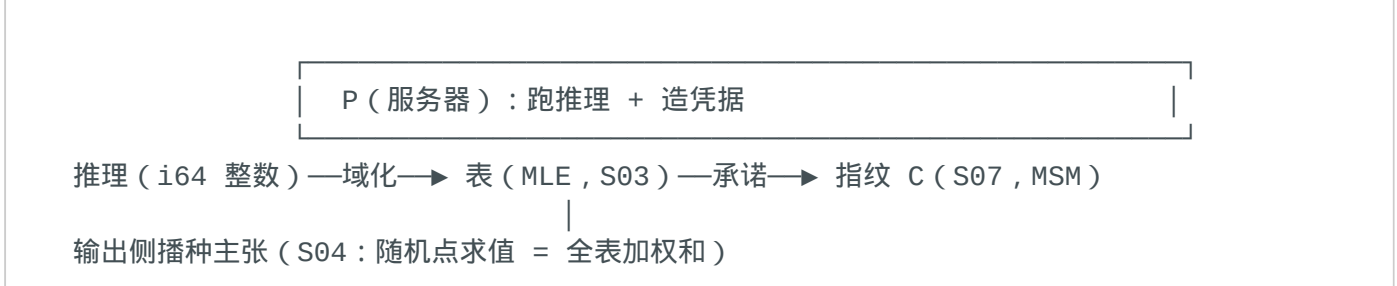
S10 · 组装：一台完整的机器（GKR 范式与 DeepProve 全景）

> 基础篇的每个零件都造完了。这一讲做三件事：①把 S01 的"审计员游戏"正式结案——画出整台机器，每个部件标注它是哪一讲造的；②给"层与层如何接力"起名字（GKR 范式）；③把镜头拉远，看这台机器在 DeepProve 里如何驱动一次真实的 LLM 推理证明。 前置：S01–S09 全部。本讲之后，你具备读懂系统主教材《prove_chunk_原理与实现详解.md》的全部语言。

---

1. 结案：整台机器

回看 S01 的三个要求，逐条对账：





三个要求全部兑现，而且每个箭头你都亲手算过。这张图就是全部理论——剩下的内容都是工程放大。

2. 层与层的接力：GKR 范式

图中最值得命名的一环是"逆拓扑逐算子归约"。它的学理出身是 **GKR 协议**（Goldwasser–Kalai–Rothblum, 2008）：把计算按层切分，**用一次 sumcheck 把"第 i+1 层输出在某随机点的取值"归约成"第 i 层输入在某随机点的取值"**，从输出层走到输入层，验证方始终只追踪 O(1) 条主张。

DeepProve 论文对此的表述：*"Similar to the fabled GKR protocol, we compose these operator protocols in a layer-by-layer fashion."* 工程上有三处改造（对照你已有的知识就能懂）：

1. **层内协议按算子定制**：线性层直接用 einsum sumcheck（Σ_收缩维 LHS·RHS，S06 的二表乘积正是它的 2×2 缩影），非线性层用查表+linking（S09）——比通用 GKR 的"加法门/乘法门"表述快得多；
2. **claim 是单链**：每个算子被设计成"输出主张 ⇒ 输入主张"的一对一归约（einsum 的 sumcheck 终点就是输入点），避免了经典 GKR 每层两条主张的合并开销；
3. **静态部分前置**：权重/查表在 setup 期就承诺好（S07 的仪式做一次、全体层共用），证明期只对动态 witness 承诺。

为什么这叫"范式"：S09 的分数折叠树、卷积的 FFT 矩阵委托（zkCNN 式蝶形分层），都是同一句话的不同应用——**"分层 + 每层一次 sumcheck + 挑战点逐层传递"**。学会识别这个形状，你就获得了读任何新协议的模板。

3. DeepProve 全景：一次真实的 LLM 推理证明

把机器开起来。以 GPT-2 的一次推理（序列长 s=512）为例，端到端数据流：

```
① 前向推理（i64 定点算术）：
   Transformer 逐层算，每个节点的输入/输出/辅助量（LN 均值方差、softmax 位移）
   记入 Trace —— 一张 HashMap<节点, 快照>。这是证明的唯一数据来源。

② 域化与承诺（S02/S03/S07）：
   每个 i64 → Fr（负数加一圈）；表补 2 的幂 → MLE；
   全部 witness 表批量 MSM 承诺（2²⁰ 一块，块数 ≈ 元素总量/2²⁰）；
   权重/查表的承诺 setup 期已做好。

③ 播种（S04/S05）：
   V（transcript）掷随机点，对"最终输出"（只有最后一行 logits！）
   与"跨块边界张量"求值立主张。

④ 逆拓扑逐算子归约（S06/S09 + 本讲 §2）：
   稠密层：einsum sumcheck（收缩维 768 → 10 轮；建表扫全张量）；
   GELU/softmax/RMSNorm：查表 witness + LogUp 双侧折叠 + linking；
   每个算子吐出输入侧新主张 → 上游继续。

⑤ 收口（S08）：
   上百条主张 → batched sumcheck → RLC → HyperKZG 折叠 + 三点 → 1 次打开。

⑥ 打包与验证：
   ChunkProof（本 chunk 的全部 sumcheck 消息 + 打开证明 + 结构与边界承诺）
   → 多 chunk 按 seed 链串联（每个 chunk 的随机性依赖前面所有 chunk 的承诺
     ——验证方逐字节重放，chunk 无法被重排/替换）
   → V 核对：每轮 sumcheck（便宜）、边界承诺相等（相邻 chunk 对同一张
     边界张量的两次独立承诺必须相同）、配对核验（1 次 MSM + 1 次配对）。
```

两条成本规律（现在你可以自己推出来，不许背）：

- **MSM 的量 ∝ 数据总量**：witness 域化元素有多少，就要做多少 MSM 标量乘（每 2^{20} 一块，一块 ≈ 2.7×10^7 次点加，Pippenger w=10）。注意力 score 张量是 s×s 的 → 数据量随序列长度平方涨；
- **sumcheck 的实例数 ∝ 算子节点数**，单实例的轮数 ∝ log(收缩维)，但**建表段扫全张量** → 总时间仍随数据量涨，只是涨的方式不同（带宽型 vs 实例数）。

final-token 优化的定量含义：GPT-2 的 logits 是 512×50257 的表（i64 下 206 MB），但用户只关心**最后一个 token** 的输出——只对最后一行（0.4 MB）立主张、进证明。证明量从 $O(s^2)$ 降为 $O(s)$。

chunk 与流式：一张 2^{20} 的表 = 32 MB（Fr），GPT-2 一层几十张——内存装不下整份 trace 时，把模型图切成 chunk：每个 chunk 证明完立即释放其张量（流式模式，本仓库当前分支的主题）；chunk 间靠 §3-⑥ 的两个机制焊死。**你在 S08 学的收口在每个 chunk 内部各跑一次**。

4. 读代码的地图（现在每个路标都认识）

`zkml/src/iop/prover.rs:588`	`prove_chunk` —— §3 ②~⑤ 的全部本体
`zkml/src/iop/streaming.rs:229`	流式协调（chunk 完成即证明）
`zkml/src/layers/einsum/`	线性层协议（S06 的放大版）
`zkml/src/lookup/`	查表层（S09 的放大版）
`zkml/src/poly_commit/`	收口三步（S08 的放大版）
`dp-crypto/sumcheck/`	sumcheck 引擎（S06 的引擎）
`dp-crypto/arkyper/`	承诺与打开（S07/S08 的引擎）
`zkml/src/iop/verifier.rs:755`	验证方重放（§3-⑥）

主教材：《prove_chunk_原理与实现详解.md》——你现在拥有读懂它 100% 行内术语的全部前置。阅读任务：全文精读，每个 文件:行号 点开验证，整理"讲义没覆盖到的三处实现细节"提交。

5. 动手作业（P8，毕业前最重的一次）

1. 画出"稠密 → GELU → 稠密 → logits"四节点模型在 DeepProve 里的**完整**证明数据流：标出每张被承诺的表、每条主张的诞生点与消费点、每次 transcript 采样、收口三步——全部标注 文件:行号。
2. 跑通 GPT-2 端到端（入口见 `L07 §0`），记录：证明耗时、验证耗时、proof 大小、chunk 数；用 §3 的两条规律解释你看到的数字。
3. 白板抽查准备：导师将从 {sumcheck 健全性（S06 §5）、三点恒等式推导（S08 §4）、LogUp 恒等式（S09 §3）、batched sumcheck 的收口恒等式（S08 §2）} 中抽一个，要求 10 分钟白板讲清。

做完放入 `homework/s10/`。**这门课的"理论部分"到此结束**——你已经站在能读论文、读引擎、改协议的位置上；后续路线（论文精读、期末项目、前沿专题）见 `L08`/`L09`。

S11 · 论文精读：DeepProve（逐节伴读）

> 配套论文：`zkml_bootcamp/deepprove.pdf`（34 页；正文 p.1–13，附录 p.14–34）。 用法：**左手 PDF、右手本讲义**，逐节对照。每节给你四样东西：原文说了什么（忠实摘要，标页码）→ 通俗解释（接 S01–S10 的钟表世界语言）→ 与课程哪一讲对应 → 容易读错的地方。 先修：S01–S10 全部。没有这些前置，§4 的每个公式都会退化成名词堆叠；有了它们，这篇论文应该读起来像"复习"。

---

0. 论文的全局地图（先看这个再进正文）

```
p.1-3    §1  Introduction —— 为什么"端到端验证 LLM 推理"是新的、难的（5 大挑战）
p.4      §2  Preliminaries —— 量化公式、LLM 结构
p.4-5    §3  Technical Overview —— 全文纲领：认证算法 / 纯整数推理 / 高效证明
p.5-10   §4  DeepProve —— 系统设计：预处理 → 证明 → 逐算子协议（含 4 个"妙手"）
p.10-13  §5  Evaluation —— 表 1-8 的正确读法
p.13-14  §6  Related Work —— 与 zkML/zkGPT/zkTorch 的定位
p.16+    Appendix A/B —— Protocol 1-14（数学细节，课程已覆盖大半）
```

读法建议：第一遍只读 §1.1、§3、§4.1–4.3.2、§5 的表 1/3/7（这些是论文的"主张 + 主菜"）；第二遍补 §4.3.3–4.5；附录在需要核对公式时再查。

---

1. Abstract & §1 Introduction（p.1–3）：论文在争什么

原文主张：DeepProve 是**第一个**对"完整 LLM 推理"做端到端验证的系统——验证**所有生成 token**。此前的工作只覆盖一部分：zkGPT 只证**单个 token**，zkLLM 只证 **Softmax** 这一个算子，zkML 只证特定模型的特定矩阵乘。

通俗解释：想象你怀疑云服务商"说用大模型、实际用小模型糊弄你"（论文给了三个场景：企业降级模型省成本、编程助手执行恶意代码、政府采购防腐败）。SNARK 能解决——但此前没有任何系统能把"一整段生成文本确实由该模型算出"完整证下来。
DeepProve 的核心断言：**证"输出序列正确"，而不是把推理电路化**。

关键数字（摘要原话，记这几个就够）：

- 吞吐 **174 TPM**（GPT-2）/ **86 TPM**（Gemma 3），单机；
- 比当时最优系统 [zkGPT] 快 **20–60×**；
- 验证 **1–3 秒**；证明 7–27 MB（GPT-2）、16–54 MB（Gemma 3）；
- 分布式 17 台机器 **1655 TPM**。

§1.1 五大挑战（论文诚实列出的"为什么以前没人做成"）——每条都对应你课程里的一个知识点：

挑战（原文）	人话	课程对应
Quadratic blowup（p.2）	attention 让单 token 电路已是平方级；逐 token 建电路的证明时间是 O(|C|³)——"prohibitively impossible"。推理界用 KV-cache 逃掉了，但证明世界里 P 要证的是"算过"，cache 反而是要被证明的数据	S10 §3 的 final-token 优化就是答案
Pure-integer inference（p.2）	SNARK 活在素数域，LLM 活在浮点；浮点上域有精度/效率损失；量化格点又必须够细——**格太粗，小值装不下，离群值撑爆格**	S02 域化；S09 requant 表
Outlier activations（p.2）	激活里混着 [9,1,33,11,3,**1000.0**] 这种离群值：8-bit 量化会把它们压成 0 或 1，精度归零	S09 §1"非线性为什么难"的量化侧面
Requantization strategy（p.2-3）	新式 SNARK 用 64-bit 小域（快），但量化乘几次就位长爆炸，必须"每几步 requant 一次"，而 requant 本身有损	S09 requant 表（乘子+右移折进查表）
Batch proving（p.3）	非线性层是"SNARK-unfriendly"且**反复出现**；整个 trace 有"多条多项式、多个求值点、多趟前向"——不批量就死	**S08 整讲**就是答案

§1.2 六条贡献（p.3）——注意每条背后是一个课程概念：

1. 专用 PIOP：requant、**用"非确定性值"证归一化层（证明方不现场算平方根和逆）**、多头 attention、旋转位置编码；轮到轮健全性 + Fiat–Shamir（= S05+S06）；
2. **认证算法**：证"整段输出"的窍门（见 §3.1 伴读）+ PCS 组装；
3. 工程优化：延迟 requant、离群钳位；
4. Rust 开源实现 + 全部数字；
5. 精度可比（12-bit 量化与浮点基线差 0.34% 以内）；
6. **proof-of-non-hallucination**：把"没幻觉"也证明出来（§4.4）。

容易读错的地方：摘要的"174 和 86 TPM"——174 是 GPT-2 且来自**次机**（表 2），86 是 Gemma 3（表 2）；主机的数字看表 1（GPT-2 135 TPM@512）。不同表不能混着比。

---

2. §2 Preliminaries（p.4）：量化公式——S09 那张表的出生证明

原文：仿射量化 x̂ = s×(x − z)（s 尺度、z 零点）；线性层 Z = X×Y + W 的量化关系 [x̂]×[ŷ] = M×[ẑ]，其中 M = (s_X·s_Y)/s_W；再把 M 写成 M = 2^(−2n)·M′（M′ 整数），M ∈ (0,1) 取 M = 2^(−k)（最接近的 2 的幂）。

通俗解释：定点乘法后，结果带着"多乘了一次的尺度"，要把它除回去。除以任意数在钟表世界里要逆元（贵），所以选 **M 是 2 的负幂 → 除法变右移**。而"右移 + 钳位"没有多项式表达 → **查表**（S09）。所以这个 §2 的三行公式，就是 S09 requant 表的出生证明。

§2.2：LLM = embedding 层 + N 个 transformer block + unembedding 层；自回归生成。图 1 是 GPT-2 结构（论文后文所有协议对着这张图讲）。

---

3. §3 Technical Overview（p.4–5）：全文纲领，三步走

3.1 认证算法——论文最聪明的一步（p.4）

原文：认证算法把**整段输入 token + 整段输出 token 拼接**后喂进模型，做**一次**前向推理；用 Attention Mask 保证位置 t 的预测只看得到前 t−1 个 token；逐位置核对"模型预测 == 声称的下一 token"；最后一个位置才真正做一次推理步。

通俗解释（用论文自己的例子，p.4）：prompt 是 "The cat"，声称输出 "sat"。拼成 [The, cat, sat] 喂进模型一次，模型在每个位置给出预测 [cat, sat, on, …]。核对：位置 0 预测 "cat" √、位置 1 预测 "sat" √（**位置 1 只能看到 "The cat"——Attention Mask 保证它没用过 "sat" 这个答案**）。

为什么这一步是革命性的：它把"逐 token 证 s 次"（平方级）换成"一次前向 + 逐位置核对"（线性）——这正是 §1.1 第一条挑战的解。依赖的前提：**argmax 的确定性**（论文：GPT-2 图 1 与 Gemma 3 图 4 的最后一步 argmax 提供确定性）与 Attention Mask 的可证明性（PIOP split 里它占 3.96%，表 7）。

3.2 纯整数推理的两个武器（p.4–5）

- **混合精度（mixed-precision）**：残差层把"归一化后的输入"和"尺度千差万别的块输出"相加——两者必须同一尺度 → 中间用**更大的位宽**，让量化格细到能同时罩住大小两端的值。代价（精度损失）用更大的格换回来。
- **离群平滑（outlier smoothing）**：用正交矩阵 M（$M \cdot M^T = I$）：$X \cdot W = (X \cdot M) \cdot (M^T \cdot W)$。妙处：**X·M 和 M^T·W 都比原 X、W 好量化**（离群值被正交旋转摊匀）；且 X 永远不需要真正物化——右乘 M 直接**吸收进 W**（$W' \leftarrow W' \cdot M$）。论文引用 [CCKDS23]。

与课程对应：这两条是**推理算法设计为可证明性服务**的例子——量化的每个选择（位宽、平滑、钳位）都在为"查表/sumcheck 能处理"让路。课程 S09 §1 讲的"非线性难"，论文在这里给了系统级答案。

3.3 高效证明（p.5）

四个决定，全部是课程已学概念的组合拳：

1. 认证算法让**一次前向产出多个预测** → 用 **batched variants** 一次证多个（S08）；
2. 工具 = **sumcheck（线性层）+ LogUp（非线性层）**，按 GKR 式逐层组装（S06/S09/S10 §2）；
3. **不承诺中间值**——只承诺模型参数和查表需要的 witness（"cannot be virtualized"，§4.2；virtual claims 概念：验证方能自己推的值不承诺，见 [Tha25]）；
4. **延迟 requant**：requant 与它后面的非线性操作**合并成一次查表**一起证——两次查表变一次。

---

4. §4 DeepProve 设计（p.5–10）

4.1 预处理（p.5–6）与 4.2 证明阶段（p.6）

- **预处理（一次性）**：①解析 safetensors/GGUF/ONNX 成算子图；②量化：**校准集**跑一遍收集每层统计 → 定每层的 s、z 与钳位范围 [−a,a]（离群值截断）；③承诺权重与量化参数（batch commit，S07）。
- **证明（每个输入一次）**：①量化推理（逐算子：钳位 → 平滑 → 混合精度 → requant），**同时记录执行 trace（= witness）**；②witness 承诺：只承诺"**不能被 virtualize**"的值——例如 softmax 前加的那些数（LogUp 要求承诺表和 witness，S09）；③算子证明。

算子证明循环（p.6，§4.3 开头）——这就是你 S10 §2 画的 claim 链，论文原文的表述：从最后一层 L 开始，P 对 L 的输出立 claim；Fiat–Shamir 吸收全部承诺；跑 PIOP；V 得到 ①L 正确的证明 ②模型参数的 claims ③witness 的 claims ④**L−1 输入的 claims（virtual claims）**；如此逆推到输入层；PCS 收口时用 batching opening 把所有模型/witness 的求值合并成一**次**打开（S08 整讲）。

4.3.1 Embedding（p.7）——第一个"妙手"

笨办法：查表论证表 T（词表 5 万 × 维度 768 的表！）——承诺量爆炸。论文的观察：**验证方手里就有索引向量 x**（输入是公开的），于是把 Embed 重写成矩阵乘 Y = H_T·X，其中 H_T 是每行恰有一个 ±1 的**稀疏矩阵**。妙处：H_T 非零元是 ±1 → **证明方根本不用承诺它的 MLE**，验证方本地 O(s·log n) 就能算它的值。**教学点：证明方不承诺验证方能自己算的东西——S10 §3-⑥ 的 verifier 重放、S08 里 eq 表不用承诺，全是同一个原则**。

4.3.2 Requantization（p.7–8）——第二个妙手（对应 S09 §4.3.2 的完整版）

两种情况：**Case 1**（x 在正确范围内）：y = ⌊c′·x/2^(δ+f)⌋ − 2^(g−1)——除法是 2 的幂 → 右移 → 用**可分解表**（把 x 切成 k-bit 块，逐块范围检查，只保留最高 g/q 块）。**Case 2**（x 超范围）：先算 x′ = Quant(2^k/x + 2^(g′−δ−1))——**用查表实现"取倒数"**！然后查三张表：T_a（范围检查）、T_b（符号判别：负 → −1、正 → 1）、InRange（高位块是否全零），组合出钳位后的输出。全部是查表——**没有除法、没有开方、没有比较门**。

课程对应：S09 §4.3.2 的"requant 的乘子与右移已折进表里"就是这里的完整版。这是全文**最贵的一个算子**——§5 表 7 显示 requant 占总证明时间 **27%**。

4.3.3 RMSNorm（p.8）——第三个妙手（"证明方送答案"模式）

标准做法要证明"除以 sqrt(ΣX²)"——开方+逆元都是 SNARK-unfriendly，且每步引入舍入误差。论文的做法：

1. 证明方**自己带一个非确定性 witness**：d[i] = Quant(1/√(1/d·ΣX[i,d]²))——即"把倒数直接算好送给验证方"；
2. 证 Y[i,j] = X[i,j]·d[i]（一个普通乘法 sumcheck）；

3. 但验证方不知道 d 是不是真的——所以**再加一条完备性检查**：$\Sigma_j Y[i,j]^2 = n\cdot(\Sigma X^2)/(\Sigma X^2) = \mathbf{1}$。

教学点：这是"证明方送答案 + 检查答案"模式——把贵计算（开方/逆）从电路里挪到证词里，再用一条**便宜的代数恒等式**验证答案的自洽。注意恒等式的形状：分子分母同项相消（论文原式 = $n\cdot(\Sigma X[i,j]^2)/(\Sigma X[i,j]^2) = 1$），你还能看出它为什么防得住"随便编一个 d"：d 错了则 $Y = X\cdot d$ 错，$\Sigma Y^2 \neq 1$。Protocol 10（附录 B）含 requant 变体。**RMSNorm 占 witness commit 的 10.59%、PIOP 的 6.02%（表 7）**。

4.3.4 LayerNorm（p.8）

LayerNorm(X) = RMSNorm(X − μ)，所以拆成两步：①对 $X' = X - \mu\cdot I$ 跑 RMSNorm 协议；②μ 本身也是非确定性的，其良构性用**一次查表**验证：$\text{Quant}(1/d\cdot\Sigma X[i,j]) \equiv \mu_i$。又是一个"均值倒数查表"。

4.3.5 MHA（p.8–9）——第四个妙手

单头 attention（Protocol 14, B.8）：P 先算 $T_K = Q\cdot K^T$ 与 $T_V = \text{Softmax}(T_K/\sqrt{(d/h)})$，发 Y 的承诺；V 得到 Y 在随机点 r 的求值主张后，**逐步反推**：先从 f_V 拿到 V 的主张（$y_i = \Sigma_x \text{eq}(x,r_y)\cdot v_x$），对 T_K 跑 softmax 的查表协议（Protocol 13, B.7），再对 $T_K = Q\cdot K^T$ 跑**批量矩阵乘 sumcheck**（Protocol 4），最后用矩阵乘 PIOP 证 $Q = X\cdot W_Q$（K、V 同理）[Tha13]。**三个矩阵乘 + 一个查表，串成一条链**——你的 claim 链直觉直接适用。

d/h 不是 2 的幂怎么办：pad 到 $\hat{Q},\hat{K},\hat{V} \in R^{(s\times\hat{d}_h)}$，pad 矩阵 $P_I \in \{0,1\}^{(d\times\hat{d}_h)}$ 每列恰一个非零元 → **V 用 O(d) 就能本地评估它**——又是"稀疏=不承诺"原则。

4.3.6 非确定性（p.9）

LLM 推理可能是**随机的**（从 top-k 采样）。解法：把"纯随机种子"当附加输入——种子正确性的证明用可验证随机数或与 V 的交互协议（§4.3 [CCH+19]）；论文强调这是**先前所有可验证 LLM 推理都没处理**的问题。

4.4 非幻觉证明（p.9–10）——额外的赠品

LLM-Check [SBS*24] 的检测信号：Attention Score′ = $\Sigma_i \log \text{Ker}_i$，其中 Ker(Q,K) 经因果掩码后是**下三角矩阵 → 行列式 = 对角元之积** → log-det = 对角线 log 之和。

DeepProve 用**查表近似 log + sumcheck 求和**把它并入电路（Protocol, B.9）。结果（表 5，512 长度）：GPT-2 AUROC 68.69 / 准确率 67.90 / **只花 98.1 ms**；Gemma 3：65.94 / 64.16 / 21.1 ms——与原方法（非量化、非证明版）的 AUROC 接近（原 LLM-Check：GPT-2 68.29%）。**"证明没幻觉"只占证明总时间的千分之一量级**——因为它的查表/求和形状和主体协议完全同构。

4.5 分布式证明（p.10）

chunk 切分 DAG；每个 chunk 是独立子 DAG；**三步（witness 承诺/证明主张/承诺打开）都能在子图上做且不重做密码学**；每个 worker 把 chunk 的**输入和输出都承诺**（V 核对：①切分与原图一致、没丢节点；②本 chunk 输入承诺 = 下一 chunk 输出承诺——**你 S10 §3 的"边界承诺相等"正是论文原话的机制**）；coordinator 把 trace 子集异步分发（某个 chunk 的 trace 一就绪就发）——除了最后一个 chunk，通信延迟被流水线藏掉。数字：9 台 1079 TPM（6.19×）、17 台 1655 TPM（10.64×）；通信总量 43–54 MB；100 Mb/s 广域网下只慢 2%。

---

5. §5 Evaluation（p.10–13）：教你读表 1–8

机器与实现：主机 EPYC 9534（24 核 2.9 GHz，500 GB）；次机 Ryzen 7950X3D（16 核 4.2 GHz，128 GB）。实现：Rust + Scroll 的 **Ceno** 库；PCS 两套——**BaseFold**（透明、抗量子、证明大）与 **HyperKZG**（小证明、需 setup）——**PIOP 与 PCS 解耦，两套共用同一套算子协议**（你在 S07/S08 学的机制）。

表 1/2（性能主表）怎么读（GPT-2, 512 长度为例）：

PCS	证明	验证	证明大小	TPM
BaseFold（主机）	41.98 min	4.44 s	24.33 MB	135.26
HyperKZG（主机）	76.7 min	4.02 s	**11.4 MB**	122.25

- **证明时间随序列长度近似线性**（长序列摊薄更好 → TPM 上升）；
- HyperKZG 证明**小一半**但证明**更慢**（对 witness 的承诺效率随长度此消彼长——论文原话："HyperKZG being more efficient to commit to witness polynomials, which become more relevant at longer sequence lengths"）；
- Gemma 3 比 GPT-2 慢：模型大 2 倍余；Gemma 3 在次机 512 长度 HyperKZG **OOM**（表 2 里标 OOM）；

- **吞吐的 CPU 时钟故事**：次机核少但时钟快（4.2 vs 2.9 GHz）→ 次机反而 175 TPM——**吞吐瓶颈在单线程延迟而非核数**。

表 3（与先前系统对比）：zkGPT 275 TPM 但**只证单 token** 且用 200 GB；zkTorch 4 TB RAM；Ours 127 TPM 证**全部输出**——"数字不能裸比，证的东西不同"（论文自己先说了这一点，并注明 zkLLM 因 GPU 不可复现被排除）。

表 7（逐层瓶颈，p.13）——全文最有行动价值的一张表：

- 证明时间构成：witness 生成 ~57% / PIOP ~35% / 打开 ~1%；
- **>50% 花在激活函数、Softmax、requant 上；requant 单项最贵 = 27%**——查表论证是整个系统的主战场（S09 的地位由此而来）；
- witness commit 侧：softmax 29.68%、requant 27.49%、activation 22.96%（GPT-2）；Gemma 3 里 requant 41.60% 更重。

验证与证明大小：验证 <3.2 s（GPT-2）/<3.5 s（Gemma 3）随长度略涨；证明 9–43 MB，HyperKZG 系统性小于 BaseFold。

内存：GPT-2 9 GB / Gemma 3 176 GB（后者在 128 GB 次机 512 长度 OOM）——>3B 参数模型需要 TB 级内存，空间高效 PCS [FP24,FP25] 或磁盘化是未来方向。

精度（表 4）：cosine 相似度——GPT-2：8-bit 0.9967、10-bit 0.9993、12-bit 0.9996；**Gemma 3 必须用 12-bit**（8-bit 只有 0.2935、10-bit 0.4998——它有极端离群值 + 额外 RMSNorm 层，传统平滑技术不适用）。量化从 8→12 bit 只增加 **0.8% 证明开销**（因为 requant 的查表协议不敏感于位宽——延迟 requant + 分解表的设计红利）。

非幻觉（表 5）：AUROC GPT-2 68.69（原 LLM-Check 68.29）、Gemma 3 65.94；prover 时间 98.1/21.1 ms——"证明中的千分之一"。

分布式（表 6）：9 台 6.19×、17 台 10.64×（512 长度 1655 TPM）；通信总量仅 43–54 MB（一轮交互）；广域网 100 Mb/s 下只慢 2%。

---

6. §6 Related Work + 附录地图（p.13–16）

定位一句话：经典 ML 可验证（决策树/小网络）→ 常规可验证推理（CNN，zkCNN 一脉 [LXZ*21] 精神最接近）→ 通用 ML 框架（性能换通用性）→ LLM 专用（zkML/zkTorch/zkGPT）。对三个直接竞品的判词（论文原话的转述）：

- **zkML**：把 GPT-2 编进 Halo2 电路——"generic"的代价是效率塌方 + 巨型架构要求；

- **zkTorch**：中间路线（按算子分解），但只证一**个输出 token**，实验用 64 核 4 TB RAM，且 sealing factor 限 2 的幂（精度损失）；
- **zkGPT**（SOTA）：单 token，且改掉了 GPT-2 的残差连接/attention mask/输出投影层；定点乘 + 固定 scale 查表 requant **无钳位** → 表更大。**DeepProve 是唯一在"证明的同一代码库"上给出精度测量的工作**（附录 C 列了量化/查表选表/钳位的各种坑）。

附录地图（配合课程）：

- **A.1 Protocol 1（sumcheck）**：就是 S06，论文给了轮到轮健全性（κ_rbr = min_j d_j/|F|）的正式定义与"domed state"证明——S06 §5 的"死亡螺旋"就是论文里 leave-the-dome 的过程；
- **A.2 Protocol 2（batched sumcheck）**：注意——论文版是"多个 sumcheck 实例用随机权重 r^j **线性组合**、最终在**公共点 r** 求值"；代码里跨不同点用的是 **eq 表进 sumcheck**（S08 §2）。两种批量视角互补，读的时候别把它们当成同一个协议；
- **B.7–B.14**：softmax（13）、单头 attention（14）、attention-score（B.9 非幻觉）等逐算子协议；
- **C**：量化的工程坑（钳位、选表、位宽）——§1.1 离群值挑战的工程落地。

---

7. Protocol ↔ 课程对照总表

论文	内容	课程
Protocol 1（A.1）	sumcheck + RBR 健全性	S06
Protocol 2（A.2）	batched sumcheck（同变量数实例的组合）	S08 §2（代码版：eq 表）
Protocol 3/4	矩阵乘 / 批量矩阵乘 PIOP	S06 §6 的二表乘积；S08 的 γ 批处理
Protocol 5	requantization	S09（论文 §4.3.2 是完整版）
Protocol 10	RMSNorm（含 requant 变体）	S09 §6 表 + 本讲 §4.3.3
Protocol 13	softmax	S09 §6 表
Protocol 14	单头 attention	本讲 §4.3.5
§4.1 预处理	解析/量化校准/承诺	S07 仪式 + S10 §3
§4.4	非幻觉（attention score）	本讲 §4.4

8. 自测题（P12 素材）

1. 认证算法为什么能把证明开销从 $O(s^2)$ 降到 $O(s)$？Attention Mask 在其中扮演什么角色？（§3.1）
2. 离群平滑的 $X \cdot W = (X \cdot M)(M^T \cdot W)$ 为什么"永远不需要物化 X"？如果 X 要被承诺，这个技巧还成立吗？
3. RMSNorm 的"证明方送倒数"d 有什么风险？论文用哪条恒等式封堵？（提示：$\Sigma Y^2 = 1$ 的分子分母为什么恰好相消？d 随便编一个会怎样？）
4. embedding 协议为什么连 H_T 都不用承诺？"验证方能本地算的东西不承诺"在本系统里还有哪三处体现？
5. 读表 7：为什么说"requant 是最贵算子"但"PCS 是瓶颈却不完全主导"？这两句话矛盾吗？
6. 论文 Protocol 2（A.2）与你 S08 学的 eq 表批量有何不同？各自适用于什么场景？
7. 分布式方案里，为什么 worker 要同时承诺 chunk 的输入**和**输出？少了哪一个会出什么作弊？

9. 关键数字速查（复习用）

- 吞吐：单机 174/86 TPM（GPT-2/Gemma 3，次机）；比 SOTA 快 20–60×；验证 1–3 s；证明 9–43 MB；分布式 17 台 1655 TPM（10.64×）；
- 对比基线：PyTorch 71.4 TPS vs 证明 2.8 TPS（≈24× 慢）；
- 瓶颈：requant ≈27%（最贵单项）；激活+softmax+requant >50%；PCS 主瓶颈但被 GKR 式 PIOP 摊薄；
- 精度：GPT-2 12-bit cosine 0.9996（与浮点差 0.34% 以内）；Gemma 3 必须 12-bit（8-bit cosine 仅 0.29）；量化 8→12 bit 只加 0.8% 证明时间；
- 非幻觉：AUROC 68.69/65.94，耗时 98.1/21.1 ms；
- 内存：GPT-2 9 GB / Gemma 3 176 GB；>3B 参数需 TB 级（开放问题）。

读完后：按 L08 的 P12 写论文报告（选 Gemini §2.4.2 或本论文 §4 与附录 B 的某个协议做深挖），或者直接进入 W9 的代码精读——论文里每一个"we commit / we don't commit"的决定，你都能在 `prove_chunk_原理与实现详解.md` 里找到对应的代码行。